OneTrans：在工业推荐中使用一个 Transformer 统一特征交互与序列建模

Zhaoqi Zhang*
Nanyang Technological University
ByteDance
Singapore, Singapore
zhaoqi.zhang@bytedance.com

Haolei Pei*
ByteDance
Singapore, Singapore
haolei.pei@bytedance.com

Jun Guo*
ByteDance
Singapore, Singapore
jun.guo@bytedance.com

Tianyu Wang
ByteDance
Singapore, Singapore
tianyu.wang01@bytedance.com

Yufei Feng
ByteDance
Hangzhou, China
fengyihui@bytedance.com

Hui Sun
ByteDance
Hangzhou, China
sunhui.sunh@bytedance.com

Shaowei Liu†
ByteDance
Singapore, Singapore
liushaowei.nphard@bytedance.com

Aixin Sun†
Nanyang Technological University
Singapore, Singapore
axsun@ntu.edu.sg

Abstract

在推荐系统中，扩大特征交互模块（例如 Wukong、RankMixer）或用户行为序列模块（例如 LONGER）的规模，已经取得显著成效。然而，这些工作通常沿着独立路线推进，不仅阻碍双向信息交换，也妨碍统一优化与规模扩展。本文提出 OneTrans，一种能够同时进行用户行为序列建模和特征交互的统一 Transformer 骨干。OneTrans 使用统一 token 化器，将序列与非序列属性转换为单一 token 序列。堆叠的 OneTrans 模块在相似的序列 token 之间共享参数，同时为非序列 token 分配 token 专属参数。借助因果注意力和跨请求 KV 缓存，OneTrans 能够预计算并缓存中间表示，显著降低训练与推理的计算成本。工业规模数据集上的实验结果表明，OneTrans 能够随参数量增加而高效扩展，持续优于强基线，并在在线 A/B 测试中带来 5.68% 的人均 GMV 提升。

CCS Concepts

• **Information systems** → **Information retrieval**; **Recommender systems**;

Keywords

推荐系统, 排序模型, 缩放定律

ACM Reference Format:

Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2025. OneTrans：在工业推荐中使用一个 Transformer 统一特征交互与序列建模. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 8 pages.

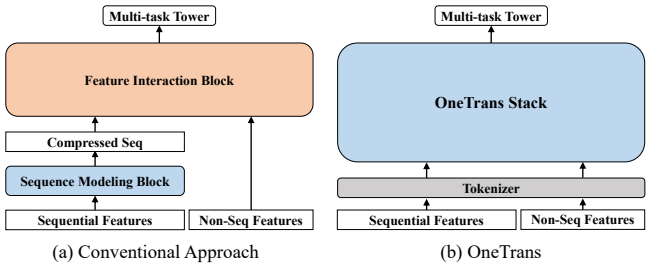


Figure 1: 架构对比。(a) 传统的先编码后交互流水线先编码序列特征，再与非序列特征合并，随后送入独立的特征交互模块。(b) OneTrans 在单一 OneTrans（Transformer 风格）堆叠中，对序列与非序列特征进行联合建模。

1 引言

推荐系统（RecSys）在电子商务 [9, 35]、流媒体 [2, 20, 28] 及社交网络 [31, 32] 等各类信息服务中发挥着基础性作用。工业推荐系统通常采用级联排序架构 [6, 16, 22]。首先，召回阶段从十亿级物品库中选出数百个候选 [13, 36]。随后，排序阶段对每个候选打分，并返回排名前 k 的物品 [11, 27, 28, 32, 37]。深度学习推荐模型（DLRM）[19] 被广泛用于工业推荐系统的排序阶段。

本文遵循 DLRM 风格的排序范式，重点研究排序阶段。在排序方面，主流方法分别对两个独立模块进行迭代：(a) *序列建模*，使用局部注意力或 Transformer 编码器，将用户多行为序列编码为候选感知表示 [1, 14, 25, 35]；以及 (b) *特征交互*，通过因子分解、显式交叉网络或特征组上的注意力，学习非序列特征（例如用户画像、物品属性和上下文）之间的高阶交叉 [11, 12, 27, 37]。如图 1(a) 所示，这些方法通常先将用户行为

*这些作者贡献相同。
†通讯作者。

This work is licensed under a Creative Commons Attribution 4.0 International License.
Conference acronym 'XX, Woodstock, NY
© 2025 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-XXXX-X/18/06

已被 The Web Conference 2026（WWW 2026）接收，最终定稿版本即将发布。

编码成*压缩的*序列表示，再将其与非序列特征拼接，随后使用特征交互模块学习更高阶的交互；我们将这种设计称为*先编码后交互*流水线。

大语言模型（LLM）的成功表明，扩大模型规模（例如参数规模、训练数据）能够带来可预测的性能提升 [15]，这启发了推荐系统领域的类似研究 [1, 32, 37]。在特征交互方面，Wukong [32] 将因子分解机模块与线性压缩相结合，通过堆叠捕获高阶特征交互，并建立了缩放定律；RankMixer [37] 则通过硬件友好的 token 混合与 token 专属前馈网络（FFN），实现了良好的规模扩展效果。在序列建模方面，LONGER[1] 将因果 Transformer 应用于用户长历史，表明增加深度和宽度能够带来单调的性能提升。尽管这些方法在实践中有效，但将序列建模和特征交互分离为独立模块，会带来两项主要局限。首先，先编码后交互流水线限制了双向信息流，使静态特征和上下文特征难以充分影响序列表示 [30]。其次，模块分离导致执行过程碎片化并增加延迟；相比之下，单一 Transformer 风格的骨干能够复用 KV 缓存、内存高效注意力和混合精度等 LLM 优化技术，从而实现更有效的规模扩展 [11]。

本文提出 **OneTrans**，这是一种采用统一 Transformer 骨干的创新架构范式，能够联合进行用户行为序列建模与特征交互。如图 1(b) 所示，OneTrans 在统一骨干内部实现双向信息交换。它使用统一 token 化器，将*序列*特征（多种行为序列）和*非序列*特征（静态用户/物品特征及上下文特征）转换为单一 token 序列，再交由金字塔式堆叠的 OneTrans 模块处理；该模块是针对工业推荐系统定制的 Transformer 变体。与 LLM 中仅包含文本的 token 不同，推荐系统的 token 来源多样。为适应这一特点，每个 OneTrans 模块采用类似 HiFormer [11] 的*混合*参数化方式。具体而言，所有*序列* token（来自序列特征）共享一组 Q/K/V 和 FFN 权重，而每个*非序列* token（来自非序列特征）均具有 *token 专属*参数，以保留各自不同的语义。

不同于传统先编码后交互框架，OneTrans 通过统一的因果 Transformer 骨干，消除了*序列*与*非序列*特征之间的架构壁垒。这一形式使推荐系统的规模扩展与 LLM 实践保持一致：通过调整骨干深度和宽度，可以扩展*整个*模型，同时无缝继承 FlashAttention [7]、混合精度训练 [17] 等成熟的 LLM 优化技术。特别地，跨候选与跨请求 KV 缓存 [1] 将时间复杂度从 $O(C)$ 降至 $O(1)$，适用于包含 C 个候选的会话，使大规模部署 OneTrans 成为可能。

总之，我们的主要贡献包括以下四点：**(1) 统一框架**。我们提出 OneTrans，一种用于排序的单一 Transformer 骨干，配备将序列和非序列特征编码成同一 token 序列的*统一 token 化器*，以及联合进行序列建模和特征交互的*统一 Transformer 模块*。**(2) 面向推荐系统的定制**。为弥合 LLM 与推荐系统任务之间的差距，OneTrans 引入*混合参数化*方式，为多样化的非序列 token 分配 token 专属参数，同时让所有序列 token 共享参数。**(3) 高效训练与服务**。我们通过逐步裁剪序列 token 的*金字塔策略*以及跨候选复用用户侧计算的*跨请求 KV 缓存*提高效率。此外，我们采用 FlashAttention、混合精度训练和半精度推理等 LLM 优化技术，进一步减少内存与计算开销。**(4) 规模扩展与部署**。随着模型规模增大，OneTrans 呈现近似对数线性的性能提升，为真实生产数据中的缩放定律提供了证据。在线部署时，它在维持生产级延迟的同时，实现了具有统计显著性的业务 KPI 提升。

2 相关工作

DIN [35] 及其会话感知变体 DSIN [9] 等早期推荐系统，使用局部注意力学习以候选为条件的用户历史摘要，但会将每个候选对应的行为压缩成固定长度向量，从而限制长程依赖建模 [34]。SASRec [14]、BERT4Rec [25] 和 BST [4] 等自注意力方法，使每个位置都能关注完整历史，从而消除这一瓶颈，并借助双向掩码提高样本效率。近年来，随着推荐系统缩放定律 [15] 研究不断深入，LONGER [1] 采用高效注意力和便于线上服务的设计，面向超长行为历史，将序列建模推进到工业规模。然而，在主流流水线中，这些序列编码器通常仍然*独立*于特征交互堆叠，因此只能进行后期融合，而不能与静态上下文特征联合优化 [30]。

在特征交互方面，早期推荐系统依赖人工构造的交叉特征或自动化乘性交互层。Wide&Deep [5]、FM/DeepFM [3, 12] 和 DCN/DCNv2 [26, 27] 等经典模型能够高效建模低阶或阶数有界的交互。然而，近期的规模扩展研究发现 [32]，当模型堆叠了足够多的交叉层后，继续增加层数便不再有帮助：模型质量进入平台期，不再持续改善。为克服预设交叉形式的僵化限制，基于注意力的方法能够自动学习高阶交互。AutoInt [24] 学习任意阶关系，HiFormer [11] 则引入组专属投影，以更好地捕获异构、非对称交互。随着规模扩展越来越多地应用于*特征交互*模块，Wukong [32] 等大规模系统通过堆叠结合线性压缩的 FM 风格交互模块，展现了可预测的增益；RankMixer [37] 则通过并行 token 混合与稀疏 MoE，在严格的延迟预算下实现良好的规模扩展效果。然而，这些交互模块通常仍遵循*交互*范式，将交互放在独立阶段，从而阻碍其与用户序列建模的统一优化 [30]。

迄今为止，推荐系统的发展主要沿着序列建模和特征交互两条独立路线推进。InterFormer [30] 尝试通过基于摘要的双向交叉架构弥合这一差距，使两个组件能够相互交换信号。但它仍将二者保留为独立模块，而交叉架构又引入了架构复杂性和执行碎片化。缺少用于联合建模与优化的统一骨干，使得将系统作为一个整体进行规模扩展仍具有挑战性。

近期的生成式推荐器（GR）研究将推荐表述为序列转导任务，并提出 HSTU [31] 等高效长上下文骨干。这一路线与依赖丰富非序列（NS）特征的 DLRM 形成互补。

3 方法

在详细介绍方法之前，我们先简要说明任务设置。在级联工业推荐系统中，召回阶段每次为用户 u 返回一个候选集合（通常包含数百个候选物品）。随后，排序模型为每个候选物品 i 预测一个分数：

$$\hat{y}_{u,i} = f\left(i \,\middle|\, \mathcal{NS}, \mathcal{S}; \Theta\right) \tag{1}$$

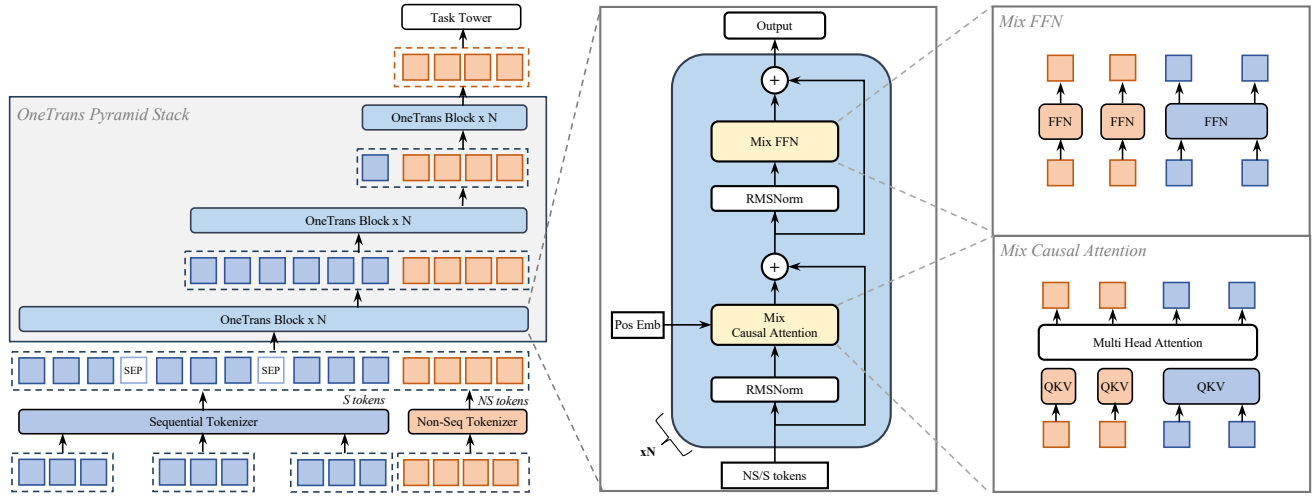
其中，$\mathcal{NS}$ 是从用户、候选物品和上下文中提取的非序列特征集合；$\mathcal{S}$ 是用户历史行为序列的集合；Θ 为可训练参数。常见的预测任务包括点击率（CTR）和点击后转化率（CVR）。

$$\begin{aligned} \mathrm{CTR}_{u,i} &= P\left(\mathrm{click} = 1 \,\middle|\, \mathcal{NS}, \mathcal{S}; \Theta\right), \\ \mathrm{CVR}_{u,i} &= P\left(\mathrm{conv} = 1 \,\middle|\, \mathrm{click} = 1, \mathcal{NS}, \mathcal{S}; \Theta\right). \end{aligned} \tag{2}$$

3.1 OneTrans 框架概览

如图 2(a) 所示，OneTrans 使用*统一 token 化器*，将序列特征 $\mathcal{S}$ 映射为 S-token，将非序列特征 $\mathcal{NS}$ 映射为 NS-token。随后，*金字塔堆叠的 Transformer* 在单一计算图中联合处理统一 token 序列。我们将初始 token 序列记为：

$$\mathbf{X}^{(0)} = \left[\text{S-tokens}\,;\, \text{NS-tokens}\right] \in \mathbb{R}^{(L_{\mathrm{S}} + L_{\mathrm{NS}}) \times d}. \tag{3}$$



(a) OneTrans Framework (b) OneTrans Block (c) Mix Parameterization

Figure 2: 系统架构。(a) OneTrans 概览。序列（S，蓝色）和非序列（NS，橙色）特征分别被转化为 token。在用户行为序列之间插入 [SEP] 后，统一 token 序列被送入堆叠的 *OneTrans* 金字塔模块，逐步缩短 token 序列长度，直至与 NS-token 的数量相同。(b) OneTrans 模块：包含 RMSNorm、混合因果注意力和混合 *FFN* 的前置归一化因果 Transformer 模块。(c) "混合"指混合参数化：S-token 共享一组 QKV/FFN 权重，而每个 NS-token 都具有 token 专属的 QKV/FFN。

该 token 序列由 L_S 个 S-token 与 L_{NS} 个 NS-token 拼接而成，所有 token 的维度均为 d。注意，S-token 中包含插入的可学习 [SEP] token，用于划分不同类型用户行为序列之间的边界。如图 2(b) 所示，每个 OneTrans 模块通过以下计算逐步细化 token 状态：

$$\mathbf{Z}^{(n)} = \text{MixedMHA}\Big(\text{Norm}\big(\mathbf{X}^{(n-1)}\big)\Big) + \mathbf{X}^{(n-1)}, \tag{4}$$

$$\mathbf{X}^{(n)} = \text{MixedFFN}\Big(\text{Norm}\big(\mathbf{Z}^{(n)}\big)\Big) + \mathbf{Z}^{(n)}. \tag{5}$$

这里，MixedMHA（混合多头注意力）和 MixedFFN（混合前馈网络）采用混合参数化策略（见图 2(c)）：序列 token 之间共享权重，而非序列 token 在注意力层和前馈层中均具有独立参数。

统一的因果掩码施加自回归约束，使每个位置只能关注其前面的 token。具体而言，NS-token 可以关注全部 S-token 历史，从而实现充分的跨 token 交互。通过堆叠这些模块，并对 S-token 采用金字塔式的尾部截取，模型逐步将紧凑的高阶信息提炼到 NS-token 中。最终 token 状态被送入任务专属预测头。

通过将非序列与序列特征整合为统一 token 序列，并使用因果 Transformer 建模，OneTrans 摆脱了传统的*先编码后交互*流水线。这种统一设计自然支持：(i) 每个行为序列内部的*序列内*交互；(ii) 多个序列之间的*跨序列*交互；(iii) 物品、用户与上下文特征之间的*多源特征*交互；以及 (iv) *序列与特征*交互，且这些交互*全部在单一 Transformer 堆叠内部完成*。

统一形式使我们能够无缝继承成熟的 LLM 工程优化，包括 KV 缓存和内存高效注意力，从而显著降低推理延迟。我们认为，这种统一形式非常适合在单一且可扩展的架构中应对多序列和跨域推荐挑战。下面详细介绍其设计。

3.2 特征与 token 化

为构建初始 token 序列 $\mathbf{X}^{(0)}$，OneTrans 首先使用特征预处理流水线，将所有原始输入特征映射为嵌入向量。随后，这些嵌入被划分为 (i) 多行为*序列*子集和 (ii) 表示用户、物品或上下文特征的*非序列*子集。两个子集分别使用独立的 token 化器。

3.2.1 非序列特征的 token 化. 非序列特征 $\mathcal{NS}$ 包括数值输入（例如价格、CTR）和类别输入（例如用户 ID、物品类别）。所有特征先进行分桶或独热编码，再映射为嵌入。由于工业系统通常包含数百个重要程度不一的特征，控制非序列 token 数量 L_{NS} 有以下两种选择：

分组 token 化器 （与 RankMixer [37] 一致）。 将特征人工划分为语义组 $\{\mathbf{g}_1, \ldots, \mathbf{g}_{L_{NS}}\}$，每组特征拼接后通过组专属 MLP：

$$\text{NS-tokens} = \Big[\text{MLP}_1(\text{concat}(\mathbf{g}_1)), \ldots, \text{MLP}_{L_{NS}}(\text{concat}(\mathbf{g}_{L_{NS}}))\Big]. \tag{6}$$

自动切分 token 化器。另一种方式是先拼接所有特征，通过单个 MLP 进行一次投影，再切分：

$$\text{NS-tokens} = \text{split}\Big(\text{MLP}(\text{concat}(\mathcal{NS})),\ L_{NS}\Big). \tag{7}$$

相比于分组方式，自动切分 token 化器通过使用单次稠密投影，减少了计算内核启动开销。我们将在实验中评估这两种选择。

最终，非序列特征的 token 化产生 L_{NS} 个非序列 token，每个 token 的维度均为 d。

3.2.2 序列特征的 token 化. OneTrans 接受如下多行为序列：

$$\mathcal{S} = \{\mathbf{S}_1, \ldots, \mathbf{S}_n\}, \quad \mathbf{S}_i = \big[\mathbf{e}_{i1}, \ldots, \mathbf{e}_{iL_i}\big]. \tag{8}$$

每个序列 $\mathbf{S}_i$ 由 L_i 个事件嵌入 $\mathbf{e}$ 构成，每个事件嵌入由物品 ID 及其对应的物品类别、价格等辅助信息拼接而成。

多行为序列的原始维度可能不同。因此，对于每个序列 $\mathbf{S}_i$，我们使用一个共享投影 MLP_i，将其中的所有事件 $\mathbf{e}_{ij}$ 转换到统

一维度 d：

$$\tilde{\mathbf{S}}_i = \left[\, \mathrm{MLP}_i(\mathbf{e}_{i1}), \ldots, \mathrm{MLP}_i(\mathbf{e}_{iL_i}) \,\right] \in \mathbb{R}^{L_i \times d}. \tag{9}$$

对齐后的序列 $\tilde{\mathbf{S}}_i$ 按以下两种规则之一合并为单一 token 序列：1) *时间戳感知*：按时间交错排列所有事件，并附带序列类型标记；2) *时间戳无关*：按事件影响程度拼接序列，例如购买 → 加购 → 点击，并在序列之间插入可学习的 [SEP] token。在后一种方式中，用户意图更强的行为被放在序列前部。消融结果表明，当时间戳可用时，时间戳感知规则优于按影响程度排序的方案。形式上，有：

$$\text{S-Tokens} = \mathrm{Merge}\big(\tilde{\mathbf{S}}_1, \ldots, \tilde{\mathbf{S}}_n\big) \in \mathbb{R}^{L_S \times d}, \quad L_S = \sum_{i=1}^{n} L_i + L_{\text{SEP}}. \tag{10}$$

3.3 OneTrans 模块

如图 2(b) 所示，每个 OneTrans 模块都是一个前置归一化的因果 Transformer，作用于*归一化后的* token 序列：前面是 L_S 个*序列* S-token，后面是 L_{NS} 个*非序列* NS-token。受异构特征组研究结果 [11] 启发，我们对 Transformer 做了轻量修改，使其支持混合参数方案，见图 2(c)。具体而言，同质的 S-token 共享一组参数；而来源和语义各异的 NS-token 则具有 token 专属参数。

不同于 LLM 输入，推荐系统的 token 序列将序列 S-token 与多样化的 NS-token 混合在一起，二者的取值范围和统计特征存在显著差异。由于这些差异，后置归一化设置可能导致注意力坍缩和训练不稳定。为避免这一问题，我们对*所有* token 使用 RMSNorm [33] 进行前置归一化，对齐不同 token 类型的尺度并稳定优化过程。

3.3.1 混合（共享/token 专属）因果注意力. OneTrans 采用带因果注意力掩码的标准多头注意力（MHA），唯一变化在于 Q/K/V 的参数化方式。令 $\mathbf{x}_i \in \mathbb{R}^d$ 为第 i 个 token。计算 Q/K/V 时，我们对 S-token（$i \le L_S$）使用*共享*投影，并使用 L_{NS} 个 *token 专属*投影处理 NS-token（$i > L_S$）：

$$\big(\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i\big) \;=\; \big(\mathbf{W}_i^{Q}\mathbf{x}_i,\ \mathbf{W}_i^{K}\mathbf{x}_i,\ \mathbf{W}_i^{V}\mathbf{x}_i\big), \tag{11}$$

其中，$\mathbf{W}_i^{\Psi}$（$\Psi \in \{Q, K, V\}$）遵循混合参数化方案：

$$\mathbf{W}_i^{\Psi} \;=\; \begin{cases} \mathbf{W}_{\mathrm{S}}^{\Psi}, & i \le L_S \quad \text{(shared for S-tokens)}, \\ \mathbf{W}_{\mathrm{NS},i}^{\Psi}, & i > L_S \quad \text{(token-specific for NS-tokens)}. \end{cases} \tag{12}$$

注意力使用标准*因果*掩码，并将 NS-token 放在 S-token *之后*。由此产生以下效果：(1) **S 侧**。每个 S-token 仅关注前面的 S 位置。对于*时间戳感知*序列，每个事件以其历史为条件；对于*时间戳无关*序列（按意图排序，例如购买 → 加购 → 点击/曝光），因果掩码使高意图信号能够为后续低意图行为提供信息并进行筛选。(2) **NS 侧**。每个 NS-token 关注*完整的* S 历史，实质上是对序列证据进行目标注意力聚合；同时，它也关注*前面的* NS-token，增加 token 级交互的多样性。(3) **支持金字塔结构**。在 S 侧与 NS 侧，因果掩码均逐步将信息集中到靠后的位置，自然支持逐层裁剪 token 的金字塔调度，后文将详细说明。

3.3.2 混合（共享/token 专属）FFN. 类似地，前馈网络遵循相同的参数化策略：NS-token 使用 token 专属 FFN，S-token 使用共享 FFN，

$$\mathrm{MixedFFN}(\mathbf{x}_i) = \mathbf{W}_i^{2}\,\phi(\mathbf{W}_i^{1}\mathbf{x}_i). \tag{13}$$

这里，$\mathbf{W}_i^1$ 和 $\mathbf{W}_i^2$ 遵循式 (12) 中的混合参数化方式，即对 $i \le L_S$ 共享参数，对 $i > L_S$ 使用 token 专属参数。

总之，与标准因果 Transformer 相比，OneTrans 只改变了*参数化方式*：NS-token 使用 *token 专属*的 QKV 和 FFN；S-token 则*共享*一组参数。单一因果掩码将整个序列连接起来，使 NS-token 能够聚合完整行为历史，同时保留高效的 Transformer 风格计算。

3.4 金字塔堆叠

如第 3.3 节所述，因果掩码将信息集中到靠后的位置。利用这一偏向近期位置的结构，我们采用*金字塔*调度：在每层 OneTrans 模块中，仅让最近的部分 S-token 发出查询，而键和值仍基于完整序列计算；查询集合随深度增加而缩小。

令 $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^{L}$ 为输入 token 列表，$\mathcal{Q} = \{L-L'+1, \ldots, L\}$ 表示尾部索引集合，其中 $L' \le L$。根据式 12，我们将*查询*修改为仅针对 $i \in \mathcal{Q}$：

$$\mathbf{q}_i \;=\; \mathbf{W}_i^{Q}\,\mathbf{x}_i, \qquad i \in \mathcal{Q}, \tag{14}$$

而键和值仍照常基于完整序列 $\{1, \ldots, L\}$ 计算。注意力计算完成后，仅保留 $i \in \mathcal{Q}$ 对应的输出，将 token 长度缩短至 L'，从而在各层之间形成金字塔层级结构。

这种设计有两个优点：(i) *逐步提炼*：将长行为历史汇集到尾部少量查询中，使模型容量集中于信息量最大的事件，并将信息整合进 NS-token；以及 (ii) *计算效率*：注意力成本变为 $O(LL'd)$，FFN 则随 L' 线性缩放。缩小查询集合能够直接减少 FLOPs 和激活内存。

3.5 训练与部署优化

3.5.1 跨请求 KV 缓存. 在工业推荐系统中，无论训练还是线上服务，来自同一请求的样本都会被连续处理：不同候选的 S-token 保持相同，而 NS-token 随候选物品而变化。利用这一结构，我们将广泛采用的 KV 缓存 [1] 整合进 OneTrans，形成统一的两阶段范式。

阶段 I（S 侧，每个请求一次）。对所有 S-token 进行带因果掩码的处理，并缓存其键/值对与注意力输出。该阶段在每个请求中仅执行一次。

阶段 II（NS 侧，每个候选一次）。对每个候选，计算其 NS-token，并对缓存的 S 侧键/值执行交叉注意力，随后通过 token 专属 FFN 层。特别地，候选专属序列（例如 SIM [21]）由于无法复用共享的 S 侧缓存，会通过池化预先聚合为 NS-token。

KV 缓存在候选之间分摊 S 侧计算，使每个候选的计算量保持较小，并消除冗余计算，从而显著提高吞吐量。

由于用户行为序列仅追加新内容，我们将 KV 缓存扩展到*跨请求*场景：每个新请求复用之前的缓存，仅计算新增加行为对应的增量键/值。这将每个请求的序列计算量从 $O(L)$ 降低为 $O(\Delta L)$，其中 ΔL 是自上次请求以来新增行为的数量。

3.5.2 统一采用 LLM 优化技术。. 我们采用 FlashAttention-2 [8]，通过分块和内核融合，减少注意力 I/O 以及普通注意力中呈二次增长的激活内存占用，从而在训练和推理中降低内存用量、提高吞吐量。为进一步缓解内存压力，我们将混合精度训练（BF16/FP16）[18] 与激活重计算 [10] 结合；后者丢弃选定的前向激活，并在反向传播时重新计算。这种组合以少量额外计算换取大量内存节省，无需改变架构即可支持更大的批次和更深的模型。

Table 1: OneTrans 实验的数据集概览。

Metric	Value
# Impressions (samples)	29.1B
# Users (unique)	27.9M
# Items (unique)	10.2M
Daily impressions (mean ± std)	118.2M ± 14.3M
Daily active users (mean ± std)	2.3M ± 0.3M

4 实验

我们通过离线评估和在线测试，旨在回答以下研究问题（RQ）：**RQ1：统一堆叠与先编码后交互的比较**。在计算量相当的条件下，*单一 Transformer 堆叠*能否带来稳定的性能提升？**RQ2：哪些设计选择至关重要？**我们对*输入层*（例如 token 化器、序列融合）和 *OneTrans 模块*（例如参数共享、注意力类型、金字塔堆叠）进行消融，以评估不同设计选择对性能和效率的重要性。**RQ3：系统效率**。在同一个 OneTrans 计算图中，金字塔堆叠、跨请求 KV 缓存、FlashAttention-2 以及结合重计算的混合精度，能否减少 FLOPs、内存和延迟？**RQ4：缩放定律**。当扩展长度（token 序列长度）、宽度（d_{model}）和深度（层数）时，损失与性能是否呈现预期的*对数线性*趋势？**RQ5：在线 A/B 测试**。在生产环境的延迟约束下，在线部署 OneTrans 能否在关键业务指标（例如 order/u、GMV/u）上带来具有统计显著性的提升？

4.1 实验设置

4.1.1 数据集. 在离线评估中，我们使用严格遵循隐私规范的生产日志（所有可识别个人身份的信息均经过匿名化和哈希处理），在大规模工业排序场景中评估 OneTrans。数据按时间顺序划分，所有特征均取曝光时刻的快照，以防止时间信息泄漏，并确保线上线下一致。标签（例如点击和下单）在与生产设置一致的固定窗口内聚合。表 1 汇总了数据集统计信息。

4.1.2 任务与指标. 我们评估式 (2) 定义的两个二分类排序任务：CTR 和 CVR。性能采用 AUC 与 UAUC（按曝光量加权的用户级 AUC）衡量。

下一批次评估。数据按时间顺序处理。对于每个小批次，我们先 (i) 在评估模式下记录预测，再 (ii) 在同一批次上进行训练。根据每天的预测分别计算当日 AUC 和 UAUC，最后对各天的结果进行宏平均。

效率指标。我们报告 *Params*（不含稀疏嵌入的模型参数量）和 *TFLOPs*（批大小为 2048 时，以 TFLOPs 计量的训练计算量）。

4.1.3 基线. 我们使用相同特征和匹配的计算预算，构建业界标准模型组合作为基线。在*先编码后交互*范式下，从广泛使用的生产基线 **DCNv2+DIN** [27, 35] 出发，逐步增强**特征交互**模块：DCNv2 → Wukong [32] → HiFormer [11] → RankMixer [37]。固定 RankMixer 后，再改变**序列建模**模块：StackDIN → Transformer [4] → LONGER [1]。

4.1.4 超参数设置。. 我们报告两种配置：**OneTrans$_S$** 堆叠 *6* 个 OneTrans 模块，宽度为 d=256，注意力头数为 H=4，目标参数量为 ≈100M。**OneTrans$_L$** 扩展至 *8* 层，宽度为 d=384。

输入经过统一 token 化器处理（多行为序列采用时间戳感知融合，非序列特征采用*自动切分*），并使用启发式金字塔调度，在各层将序列*查询* token 数量线性缩减：OneTrans$_S$ 从 1190 缩减至 12，OneTrans$_L$ 从 1500 缩减至 16。具体而言，我们逐层线性减少序列*查询* token 数量，将每层数量四舍五入到最接近的 32 的倍数，并使顶层数量与非序列 token 数量相同。

优化与基础设施。我们采用不含权重衰减的双优化器策略：稀疏嵌入使用 Adagrad（β_1=0.1、β_2=1.0），稠密参数使用 RMSProp（lr=0.005、alpha=0.99999、momentum=0）。我们采用大规模 Transformer 训练中常用的稳定化技术，包括前置归一化 [29] 和全局梯度范数裁剪 [23]。训练时，每张 GPU 的批大小设为 2048，稠密层和稀疏层的梯度裁剪阈值分别为 90 和 120。在线推理时，每张 GPU 使用较小的批大小 100，以平衡吞吐量与延迟。训练在 16 张 H100 GPU 上采用数据并行 all-reduce。

4.2 RQ1：性能评估

我们以 DCNv2+DIN 为比较基准，它是本场景在规模扩展前使用的生产基线（表 2）。在*先编码后交互*范式下，独立扩大任一组件的规模都有益：升级*特征交互*模块（DCNv2 → Wukong → HiFormer → RankMixer）或*序列建模*模块（StackDIN → Transformer → LONGER），均在 CTR AUC/UAUC 和 CVR AUC 上带来持续增益。在我们的系统中，这些指标超过 +0.1% 的提升被认为具有实际意义，而超过 +0.3% 的提升通常对应在线 A/B 测试中具有统计显著性的效果。然而，由于每个用户的样本量较小、波动较大，需要谨慎看待 CVR UAUC。

采用统一设计后，OneTrans$_S$ 相较基线在 CTR AUC/UAUC 上提升 +1.13%/ + 1.77%，在 CVR AUC/UAUC 上提升 +0.90%/ + 1.66%。在参数规模相当时，它也以近似的训练 FLOPs（2.64T 对比 2.51T）优于 RankMixer+Transformer，体现了统一建模的收益。进一步扩展后，OneTrans$_L$ 取得最优的整体提升：CTR AUC/UAUC 为 +1.53%/ + 2.79%，CVR AUC/UAUC 为 +1.14%/ + 3.23%，表明随着模型容量增长，质量表现可以预测。

总之，与独立扩展任一组件相比，将序列建模与特征交互统一于单个 Transformer 中，能够带来更可靠、计算效率更高的提升。

4.3 RQ2：通过消融研究评估设计选择

我们对提出的 OneTrans 模型开展消融研究，以量化关键设计选择的贡献。完整结果汇总于表 3。我们评估以下变体：**输入变体：** i) 用*分组 token 化器*替换*自动切分 token 化器*（第 1 行）；ii) 使用时间戳*无关*融合策略，替换时间戳*感知*序列融合（第 2 行）；iii) 移除时间戳*感知*序列融合中的 `[SEP]` token（第 3 行）；**OneTrans 模块变体：** i) 在*所有* token 之间共享一组 Q/K/V 和 FFN 参数，而非给 NS-token 分配独立参数（第 4 行）；ii) 将因果注意力替换为全注意力（第 5 行）；iii) 在*所有*层保留完整 token 序列，禁用金字塔堆叠（第 6 行）。

总之，消融实验表明：1) **自动切分 token 化器**明显优于人工将非序列特征分组为 token 的方式，说明让模型自动构建非序列 token，比依赖人工定义的特征分组更有效；2) 当时间戳可用时，**时间戳感知融合**优于基于意图的排序，表明应优先考虑时间顺序，而非事件影响程度；3) 在时间戳*无关*融合下，可学习的 **`[SEP]` token** 有助于模型区分不同序列；4) NS-token 的 **token 专属参数**优于共享投影，能够更好地区分特征；5) **因果注意力与全注意力**的性能相近，但全注意力无法使用 KV 缓存等标准优化；6) 在所有层保留全长 token 并无收益：OneTrans 能够有效地将信息汇总到尾部少量位置，因此**金字塔设计**可以安全地裁剪查询以节省计算。此外，在固定 TFLOPs 预算下，金字塔设计支持的序列长度*接近*全长设计的 1.75×，从而更充分地利用序列延长带来的收益。

Table 2: 离线效果（CTR/CVR）与效率；AUC/UAUC 越高越好。* 表示在生产环境中部署过的模型，按时间先后顺序为：*DCNv2+DIN* → *RankMixer+DIN* → *RankMixer+Transformer* → *OneTrans$_S$* → *OneTrans$_L$*

Type	Model	CTR		CVR (order)		Efficiency	
		AUC ↑	UAUC ↑	AUC ↑	UAUC ↑	Params (M)	TFLOPs
(1) Base model	DCNv2 + DIN (base)*	0.79623	0.71927	0.90361	0.71955	10	0.06
(2) Feature-interaction	Wukong + DIN	+0.08%	+0.11%	+0.14%	+0.11%	28	0.54
	HiFormer + DIN	+0.11%	+0.18%	+0.23%	-0.20%	108	1.35
	RankMixer + DIN*	+0.27%	+0.36%	+0.43%	+0.19%	107	1.31
(3) Sequence-modeling	RankMixer + StackDIN	+0.40%	+0.37%	+0.63%	-1.28%	108	1.43
	RankMixer + LONGER	+0.49%	+0.59%	+0.47%	+0.44%	109	1.87
	RankMixer + Transformer*	+0.57%	+0.90%	+0.52%	+0.75%	109	2.51
(4) Unified framework	OneTrans$_S$*	+1.13%	+1.77%	+0.90%	+1.66%	91	2.64
	OneTrans$_L$ (default)*	+1.53%	+2.79%	+1.14%	+3.23%	330	8.62

Table 3: 以 OneTrans$_S$ 模型为参照，评估输入设计与 OneTrans 模块设计选择的影响。

Type	Variant	CTR		CVR (order)		Efficiency	
		AUC ↑	UAUC ↑	AUC ↑	UAUC ↑	Params (M)	TFLOPs
Input	Group-wise Tokenzier	-0.10%	-0.30%	-0.12%	-0.10%	78	2.35
	Timestamp-agnostic Fusion	-0.09%	-0.22%	-0.20%	-0.21%	91	2.64
	Timestamp-agnostic Fusion w/o Sep Tokens	-0.13%	-0.32%	-0.29%	-0.33%	91	2.62
OneTrans Block	Shared parameters	-0.15%	-0.29%	-0.14%	-0.29%	24	2.64
	Full attention	+0.00%	+0.01%	-0.03%	+0.06%	91	2.64
	w/o pyramid stack	-0.05%	+0.06%	-0.04%	-0.42%	92	8.08

4.4 RQ3：系统效率

为量化第 3.5 节中的优化，我们在未优化的 **OneTrans$_S$** 基线上开展消融，并在表 4 中报告训练和推理指标。

如表所示，(i) **金字塔堆叠**通过裁剪序列*查询* token，同时降低训练成本（运行时间/内存）与服务开销（p99 延迟/内存）；(ii) **跨请求 KV 缓存**消除冗余序列计算，在训练和服务中均持续改善运行时间、延迟及内存用量；(iii) **FlashAttention** 带来显著的训练收益以及较小的服务改善；(iv) **结合重计算的混合精度**带来最大的服务收益（p99 延迟和推理内存），同时也提高了训练效率。

这些结果表明了 LLM 优化技术在大规模推荐中的有效性。基于上述结果，我们将模型扩展为 **OneTrans$_L$**，并表明其在线效率仍与小得多的 DCNv2+DIN 基线相当（表 5），凸显出统一 Transformer 骨干能够直接采用 LLM 优化技术的优势。

4.5 RQ4：缩放定律验证

我们沿三个维度研究 OneTrans 的*缩放定律*：(1) **长度**——输入 token 序列长度；(2) **深度**——堆叠模块数量；以及 (3) **宽度**——隐藏状态维度。

如图 3(a) 所示，增加*长度*能够引入更多行为证据，因此带来最大的收益。在*深度*与*宽度*之间，我们观察到明确的取舍：增加*深度*通常比单纯增加*宽度*带来更大的性能提升，因为更深的堆叠能够提取更高阶交互和更丰富的抽象表示。然而，更深的模型也会增加串行计算，而增加宽度更易于并行。因此，在*深度*与*宽度*之间进行选择时，应在目标硬件预算下平衡性能收益与系统效率。

我们进一步通过同时增加 OneTrans 的宽度和深度来分析缩放定律；作为比较，也将 **RankMixer+Transformer** 基线中的 RankMixer 部分扩展至 1B，然后在对数尺度上绘制 ΔUAUC 与训练 FLOPs 的关系。如图 3(b) 所示，OneTrans 与 RankMixer 均呈现清晰的对数线性趋势，但 OneTrans 的斜率*更陡*。这可能是因为以 RankMixer 为中心的规模扩展缺少统一骨干，而且其基于 MoE 的扩展主要增加 FFN 隐藏维度。综合来看，这些结果表明 OneTrans 具有更高的*参数与计算效率*，为工业部署提供了良好的性能与计算量权衡。

尽管我们能够在严格的在线 p99 延迟约束下部署 OneTrans$_L$，但在此基础上进一步大幅扩展，仍受线上效率限制；进一步的系统与模型协同优化留待未来研究。

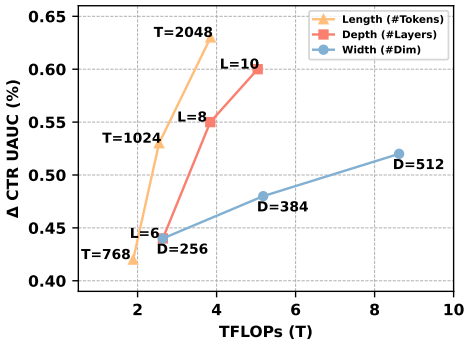
4.6 RQ5：在线 A/B 测试

我们在两个大规模工业场景中评估 OneTrans 的业务影响：(i) *Feeds*（首页信息流）；(ii) *Mall*（包含 Feeds 及其他子场景的整体场景）。流量通过哈希与用户级随机化，在用户/账号层面划分。*对照组*与*实验组*模型均使用过去 1.5 年的生产数据进行训练与部署，以确保公平比较。

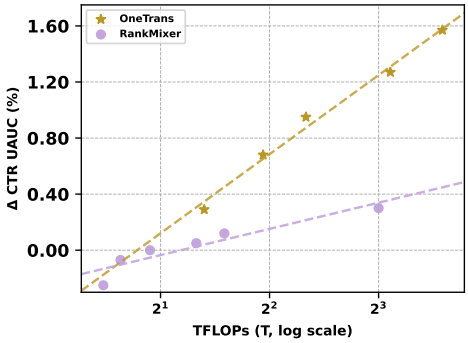
之前的生产基线 **RankMixer+Transformer** 作为*对照组*（神经网络参数量为 ≈ 100M），不使用序列 KV 缓存。*实验组*部署 OneTrans$_L$，并采用第 3.5 节所述的服务优化。

Table 4: 各变体相对于未优化的 OneTrans$_S$ 的影响。内存指标指 GPU 峰值用量。

Variant	Training		Inference	
	Runtime (ms)	Memory (GB)	Latency (p99; ms)	Memory (GB)
Unoptimized OneTrans$_S$	407	53.13	54.00	1.70
+ Pyramid stack	−28.7%	−42.6%	−8.4%	−6.9%
+ Cross-Request KV Caching	−30.2%	−58.4%	−29.6%	−52.9%
+ FlashAttention	−50.1%	−58.9%	−12.3%	−11.6%
+ Mixed Precision with Recomputation	−32.9%	−49.0%	−69.1%	−30.0%



(a) 权衡：FLOPs 与 ΔUAUC



(b) 缩放定律：ΔUAUC 与 FLOPs（对数尺度）

Figure 3: 性能权衡与缩放定律对比。

Table 5: OneTrans$_L$ 与 DCNv2+DIN 基线的关键效率指标对比。

Metric	DCNv2+DIN	OneTrans$_L$
TFLOPs	0.06	8.62
Params (M)	10	330
MFU	13.4	30.8
Inference Latency (p99, ms)	13.6	13.2
Training Memory (GB)	20	32
Inference Memory (GB)	1.8	0.8

我们报告用户级 **click/u**、**order/u** 和 **gmv/u** 相对于 *RankMixer+Transformer* 对照组的相对变化（Δ%），并给出双侧 95% 置信区间（用户级分层自助法）；同时报告**端到端延迟**，即从请求到达到响应发出这段曝光级耗时的 p99 相对变化（Δ%，越低越好）。如表 6 所示，OneTrans$_L$ 带来了稳定增益。在 *Feeds* 中，`click/u` 提升 +7.737%，`order/u` 提升 +4.3510%，`gmv/u` 提升 +5.6848%，延迟变化为 −3.91%。在 *Mall* 中，`click/u` 提升 +5.143%，`order/u` 提升 +2.5772%，`gmv/u` 提升 +3.6696%，延迟变化为 −3.26%。这些结果表明，相对于强大的非统一基线，统一建模框架在改善业务指标的同时缩短了服务耗时。

Table 6: 在线 A/B 结果：OneTrans$_L$（实验组）与 RankMixer+Transformer（对照组）的对比。Click/u、Order/u 和 GMV/u 均为相对变化（%）。延迟为曝光级端到端耗时的相对变化 Δ%（越低越好）。* 表示 $p < 0.05$，** 表示 $p < 0.01$

Scenario	click/u	order/u	gmv/u	Latency ($p99$) ↓
Feeds	+7.737%**	+4.351%*	+5.685%*	−3.91%
Mall	+5.143%**	+2.577%**	+3.670%*	−3.26%

我们还观察到用户*活跃天数*增加 +0.7478%，*冷启动商品的人均订单数*显著提升 +13.59%，凸显了所提出模型较强的泛化能力。

5 结论

我们提出 OneTrans，一种用于个性化排序的统一 Transformer 骨干，以替代传统的*先编码–再进行–交互*方式。统一 token 化器将序列与非序列属性转换为同一 token 序列；统一 Transformer 模块通过对同质（序列）token 共享参数、对异质（非序列）token 使用 token 专属参数，联合进行序列建模和特征交互。为使统一堆叠在大规模下保持高效，我们采用逐步裁剪序列 token 的金字塔调度，以及复用用户侧计算的跨请求 KV 缓存；该设计进一步受益于 LLM 风格的系统优化（例如 FlashAttention、混合精度）。在大规模评估中，随着宽度和深度增加，OneTrans 呈现近似对数线性的性能增益，并在维持生产级延迟的同时，带来具有统计显著性的业务提升。我们认为，这种统一设计为扩展推荐系统提供了一条实用途径，同时能够复用推动近期 LLM 进展的系统优化技术。

References

[1] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, et al. 2025. LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders. *arXiv preprint arXiv:2505.04421* (2025).
[2] Jianxin Chang, Chenbin Zhang, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. Pepnet: Parameter and embedding personalized network

for infusing with personalized prior information. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3795–3804.

[3] Yin-Wen Chang, Cho-Jui Hsieh, Kai-Wei Chang, Chih-Jen Lin, et al. 2010. Training and testing low-degree polynomial data mappings via linear svm. *Journal of Machine Learning Research* 11, 4 (2010).

[4] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior Sequence Transformer for E-commerce Recommendation in Alibaba. arXiv:1905.06874 [cs.IR] https://arxiv.org/abs/1905.06874

[5] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems. arXiv:1606.07792 [cs.LG] https://arxiv.org/abs/1606.07792

[6] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.

[7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.

[8] Tri Dao, Aleksander Thomas, Anima Anandkumar, Matei Zaharia, and Christopher Re. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691* (2023).

[9] Yufei Feng, Fuyu Lv, Weichen Shen, Menghan Wang, Fei Sun, Yu Zhu, and Keping Yang. 2019. Deep Session Interest Network for Click-Through Rate Prediction. arXiv:1905.06482 [cs.IR] https://arxiv.org/abs/1905.06482

[10] Audrunas Gruslys, Remi Munos, Ivo Daniel, Oriol Vinyals, and Koray Kavukcuoglu. 2016. Memory-Efficient Backpropagation through Time. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[11] Huan Gui, Ruoxi Wang, Ke Yin, Long Jin, Maciej Kula, Taibai Xu, Lichan Hong, and Ed H. Chi. 2023. Hiformer: Heterogeneous Feature Interactions Learning with Transformers for Recommender Systems. arXiv:2311.05884 [cs.IR] https://arxiv.org/abs/2311.05884

[12] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, Xiuqiang He, and Zhenhua Dong. 2018. DeepFM: An End-to-End Wide & Deep Learning Framework for CTR Prediction. arXiv:1804.04950 [cs.IR] https://arxiv.org/abs/1804.04950

[13] Junjie Huang, Jizheng Chen, Jianghao Lin, Jiarui Qin, Ziming Feng, Weinan Zhang, and Yong Yu. 2024. A comprehensive survey on retrieval methods in recommender systems. *arXiv preprint arXiv:2407.21022* (2024).

[14] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. arXiv:1808.09781 [cs.IR] https://arxiv.org/abs/1808.09781

[15] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[16] Shichen Liu, Fei Xiao, Wenwu Ou, and Luo Si. 2017. Cascade ranking for operational e-commerce search. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1557–1565.

[17] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740* (2017).

[18] Paulius Micikevicius, Sharan Narang, Jonah Alben, Greg Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *International Conference on Learning Representations (ICLR)*.

[19] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).

[20] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.

[21] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2685–2692.

[22] Jiarui Qin, Jiachen Zhu, Bo Chen, Zhirong Liu, Weiwen Liu, Ruiming Tang, Rui Zhang, Yong Yu, and Weinan Zhang. 2022. Rankflow: Joint optimization of multi-stage cascade ranking systems as flows. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 814–824.

[23] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[24] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. ACM, 1161–1170. doi:10.1145/3357384.3357925

[25] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. arXiv:1904.06690 [cs.IR] https://arxiv.org/abs/1904.06690

[26] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. arXiv:1708.05123 [cs.LG] https://arxiv.org/abs/1708.05123

[27] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of the Web Conference 2021 (WWW '21)*. ACM, 1785–1797. doi:10.1145/3442381.3450078

[28] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5249–5259.

[29] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. 2020. On layer normalization in the transformer architecture. In *International conference on machine learning*. PMLR, 10524–10533.

[30] Zhichen Zeng, Xiaolong Liu, Mengyue Hang, Xiaoyi Liu, Qinghai Zhou, Chaofei Yang, Yiqun Liu, Yichen Ruan, Laming Chen, Yuxin Chen, et al. 2024. Interformer: Towards effective heterogeneous interaction learning for click-through rate prediction. *arXiv preprint arXiv:2411.09852* (2024).

[31] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[32] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, et al. 2024. Wukong: Towards a scaling law for large-scale recommendation. *arXiv preprint arXiv:2403.02545* (2024).

[33] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in neural information processing systems* 32 (2019).

[34] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2018. Deep Interest Evolution Network for Click-Through Rate Prediction. arXiv:1809.03672 [stat.ML] https://arxiv.org/abs/1809.03672

[35] Guorui Zhou, Chengru Song, Xiaoqiang Zhu, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. arXiv:1706.06978 [stat.ML] https://arxiv.org/abs/1706.06978

[36] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1079–1088.

[37] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, Huizhi Yang, Zheng Chai, Zhe Chen, Yuchao Zheng, Qiwei Chen, Feng Zhang, Xun Zhou, Peng Xu, Xiao Yang, Di Wu, and Zuotao Liu. 2025. RankMixer: Scaling Up Ranking Models in Industrial Recommenders. arXiv:2507.15551 [cs.IR] https://arxiv.org/abs/2507.15551